

CC3074 - Modelización y Simulación

LABORATORIO

A*, Hill Climbing y Optimización de Redes

Universidad del Valle de Guatemala
2026

Propósito

Implementar los algoritmos estudiados en la unidad y aplicarlos a problemas de búsqueda, optimización local y redes. El laboratorio conserva la lógica del laboratorio base de heurísticas y redes, pero se concentra en los temas trabajados actualmente: Hill Climbing, A* y Flujo Máximo + Costo Mínimo.

Instrucciones generales

- Entregar un solo notebook (.ipynb) con código, resultados impresos, gráficas y respuestas escritas en celdas Markdown.
- Cada algoritmo debe estar implementado en una función propia y reutilizable.
- Se permite Python estándar, heapq, collections, random, itertools y matplotlib.
- No utilizar funciones o librerías que resuelvan directamente A*, Hill Climbing, Flujo Máximo o Costo Mínimo.
- Cuando se solicite mostrar el procedimiento, imprimir una tabla o registro por iteración.
- El código debe identificar claramente entradas, proceso y resultado.

Estructura del laboratorio

Parte	Tema	Problema
1	Hill Climbing	Agente viajero
2	A*	Ruta más corta en una red
3	Flujo Máximo + Costo Mínimo	Distribución de suministros

Parte 1. Hill Climbing - Problema del agente viajero

Enunciado

Un vendedor vive en la ciudad 1. Debe visitar las ciudades 2 a 7 exactamente una vez y regresar a la ciudad 1. No todas las ciudades están conectadas directamente. El objetivo es encontrar un recorrido válido con la menor distancia total posible.

Ciudad	Ciudad	Distancia
1	2	12
1	3	10
1	7	12
2	3	8
2	4	12
3	4	11
3	5	3
3	7	9
4	5	11
4	6	10
5	6	6
5	7	7
6	7	9

Modelo para Hill Climbing

Solución: un tour que inicia y termina en la ciudad 1. Función objetivo: distancia total del tour; se desea minimizar. Vecino: tour obtenido al invertir un sub-recorrido. Un vecino es inválido si utiliza una carretera inexistente.

Actividades

1. Implemente `distancia_tour(tour)`. Debe devolver la distancia total o indicar que el tour no es válido. Verifique el tour inicial 1-2-3-4-5-6-7-1.
2. Implemente `vecinos(tour)`, generando tours mediante inversión de un sub-recorrido. La ciudad 1 debe permanecer fija.
3. Implemente Hill Climbing de máximo descenso: evalúe todos los vecinos válidos y muévase al de menor distancia. Deténgase cuando ninguno mejore el tour actual.
4. En cada iteración muestre: iteración, tour actual, distancia, mejor vecino, distancia del vecino y tramo invertido.
5. Implemente una variante con reinicios aleatorios. Ejecute 100 reinicios con semilla fija y reporte el mejor tour encontrado y su distancia.
6. Genere una gráfica del mejor recorrido encontrado y explique por qué Hill Climbing puede detenerse en una solución local.

```
def distancia_tour(tour, distancias):
    pass

def vecinos(tour, distancias):
    pass

def hill_climbing(tour_inicial, distancias):
    pass
```

Preguntas de análisis

- ¿Qué representa una solución, un vecino y la función objetivo en este problema?
- ¿Por qué Hill Climbing puede detenerse aunque exista un recorrido mejor?
- ¿Qué efecto tuvieron los reinicios aleatorios?

Parte 2. A* - Ruta más corta en una red

Enunciado

Un guardabosques debe desplazarse desde la entrada O hasta el punto T. Las estaciones O, A, B, C, D, E y T están conectadas por caminos de doble sentido. Se desea encontrar una ruta de costo mínimo utilizando A*.

Camino	Distancia
O-A	2
O-B	5
O-C	4
A-B	2
A-D	7
B-C	1
B-D	4
B-E	3
C-E	4
D-E	1
D-T	5
E-T	7

Coordenadas para calcular la heurística euclidiana:

Estación	x	y
O	0.0	0.0
A	1.5	1.2
B	2.5	0.0
C	2.8	-0.8
D	5.5	1.0
E	5.2	0.2
T	9.0	0.5

Recordatorio

A* selecciona el nodo con menor $f(n) = g(n) + h(n)$. $g(n)$ es el costo acumulado desde el origen; $h(n)$ es una estimación del costo restante; $f(n)$ es el costo total estimado.

Actividades

- Implemente una función heurística(nodo, meta, coordenadas) usando distancia euclidiana.
- Implemente A* utilizando heapq. No utilizar networkx para resolver la búsqueda.
- En cada iteración muestre: nodo expandido, $g(n)$, $h(n)$, $f(n)$ y contenido de la lista abierta.
- Encuentre la ruta de O a T, muestre el camino final y su costo total.
- Ejecute nuevamente el algoritmo utilizando $h(n)=0$. Compare el resultado con A* y explique la relación con Dijkstra.
- Grafique la red con matplotlib y resalte la ruta encontrada.

```
def heuristica(nodo, meta, coordenadas):
    pass

def a_estrella(grafo, coordenadas, inicio, meta):
    pass
```

Preguntas de análisis

- ¿Qué información aporta $g(n)$? ¿Qué información aporta $h(n)$?
- ¿Por qué una heurística útil puede reducir la cantidad de nodos explorados?
- ¿Qué ocurre conceptualmente cuando $h(n)=0$?

Parte 3. Flujo Máximo y Costo Mínimo - Distribución de suministros

Enunciado

Un centro de distribución S debe enviar suministros hacia un centro de atención T. Los nodos A, B, C, D y E representan centros intermedios. Cada conexión tiene una capacidad máxima y un costo por unidad transportada.

Las capacidades conservan la red del laboratorio base. Para incorporar el tema de Costo Mínimo, en esta versión se agregan costos unitarios didácticos a cada conexión.

Tramo	Capacidad	Costo/unidad
S → A	5	2
S → B	7	1
S → C	4	3
A → B	1	1
A → D	3	2
B → C	2	1
B → D	4	3
B → E	5	2
C → E	4	1
D → T	9	2
E → D	1	1
E → T	6	3

Parte A - Flujo Máximo

- Construya la red residual incluyendo aristas de regreso con capacidad inicial 0.
- Implemente Edmonds-Karp: utilice BFS para buscar caminos aumentantes desde S hasta T.
- En cada iteración muestre: camino aumentante, cuello de botella, flujo enviado y flujo acumulado.
- Al finalizar, muestre el flujo máximo y el flujo utilizado en cada tramo.
- Verifique por código que ningún tramo exceda su capacidad y que en cada nodo intermedio el flujo que entra sea igual al flujo que sale.

```
def bfs_camino(residual, fuente, destino):
    pass

def edmonds_karp(capacidades, fuente, destino):
    pass
```

Parte B - Costo Mínimo

Ahora utilice la misma red, pero considere también el costo por unidad. El objetivo es enviar el flujo máximo obtenido en la Parte A con el menor costo total posible.

- Represente para cada arista su capacidad, costo y flujo actual.
- Implemente una red residual de costos: una arista de regreso debe permitir corregir decisiones anteriores y utilizar el costo opuesto de la arista original.
- Busque repetidamente un camino S-T de menor costo dentro de la red residual que aún tenga capacidad disponible.
- Envíe por el camino tanto flujo como permitan el cuello de botella y el flujo restante por asignar.
- Actualice capacidades residuales, aristas de regreso, flujo acumulado y costo acumulado.
- Muestre por iteración: ruta elegida, costo por unidad de la ruta, cuello de botella, flujo enviado y costo acumulado.
- Reporte al final: flujo enviado, costo total mínimo y flujo utilizado en cada tramo.

```
def camino_menor_costo(residual, costos, fuente, destino):
    pass

def flujo_costo_minimo(capacidades, costos, fuente, destino, flujo_objetivo):
    pass
```

Preguntas de análisis

- ¿Por qué maximizar el flujo y minimizar el costo son objetivos distintos?
- ¿Puede haber dos distribuciones con el mismo flujo máximo pero costos diferentes? Explique.
- ¿Para qué sirven las aristas de regreso en la red residual?
- ¿Qué conexión parece actuar como cuello de botella en la red?

Integración final

Los tres ejercicios utilizan estrategias diferentes para responder preguntas diferentes. Complete la siguiente comparación en una celda Markdown de su notebook.

Algoritmo	Pregunta principal	Información utilizada	Resultado
Hill Climbing	¿Cómo mejoro la solución actual?	Vecinos + función objetivo	Mejor solución local encontrada
A*	¿Cuál es una ruta de menor costo?	$g(n) + h(n)$	Camino y costo
Flujo Máximo	¿Cuánto puedo transportar?	Capacidades + red residual	Flujo máximo
Costo Mínimo	¿Cómo transporto con menor costo?	Capacidad + flujo + costo	Distribución y costo total

Entregables

- Un único notebook .ipynb.
- Código completo y ejecutable de los tres ejercicios.
- Resultados impresos por iteración cuando se solicite.
- Gráficas solicitadas.
- Respuestas de análisis en celdas Markdown.
- Conclusión final comparando los algoritmos.

Ponderación sugerida

Parte	Qué se evalúa	Puntos
1. Hill Climbing	Función objetivo, vecindario, algoritmo, reinicios y análisis	30
2. A*	Heurística, A*, procedimiento, comparación $h=0$ y gráfica	30
3. Flujo + Costo	Red residual, Edmonds-Karp, costo mínimo, verificaciones y análisis	30
4. Integración	Comparación y claridad de conclusiones	10
Total		100

Criterio de claridad

Las respuestas escritas deben explicar el procedimiento como si se lo contaran a una persona que no vio la clase.